



**Politechnika Rzeszowska**  
im. Ignacego Łukasiewicza

**Wydział Matematyki i Fizyki Stosowanej**

---

## **Zadanie programistyczne (Zadanie 36)**

---

**Sprawozdanie**

Kierunek: Inżynieria i Analiza Danych

Autor:

**Krystian Figiela**



Styczeń 2025

# Spis treści

|          |                                                                                                                                                                 |          |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Treść zadania</b>                                                                                                                                            | <b>2</b> |
| <b>2</b> | <b>Rozwiązanie - podejście pierwsze (brute force)</b>                                                                                                           | <b>2</b> |
| 2.1      | Analiza problemu . . . . .                                                                                                                                      | 2        |
| 2.2      | Schemat blokowy . . . . .                                                                                                                                       | 3        |
| 2.3      | Pseudokod . . . . .                                                                                                                                             | 3        |
| 2.4      | Sprawdzenie poprawności algorytmu - „ołówkowe” rozwiązanie problemu . . . . .                                                                                   | 4        |
| 2.5      | Teoretyczne oszacowanie złożoności obliczeniowej . . . . .                                                                                                      | 4        |
| 2.6      | Wykazanie poprawności algorytmu . . . . .                                                                                                                       | 5        |
| 2.6.1    | Poprawność częściowa (zastosowanie niezmiennika) . . . . .                                                                                                      | 5        |
| 2.6.2    | Własność stopu (zastosowanie półniezmiennika) . . . . .                                                                                                         | 5        |
| 2.6.3    | Wniosek końcowy . . . . .                                                                                                                                       | 5        |
| <b>3</b> | <b>Rozwiązanie - próba druga (sliding window)</b>                                                                                                               | <b>6</b> |
| 3.1      | Analiza problemu . . . . .                                                                                                                                      | 6        |
| 3.2      | Schemat blokowy . . . . .                                                                                                                                       | 7        |
| 3.3      | Pseudokod . . . . .                                                                                                                                             | 7        |
| 3.4      | Sprawdzenie poprawności algorytmu - „ołówkowe” rozwiązanie problemu . . . . .                                                                                   | 8        |
| 3.5      | Teoretyczne oszacowanie złożoności obliczeniowej . . . . .                                                                                                      | 8        |
| 3.6      | Wykazanie poprawności algorytmu . . . . .                                                                                                                       | 9        |
| 3.6.1    | Poprawność częściowa (zastosowanie niezmiennika) . . . . .                                                                                                      | 9        |
| 3.6.2    | Własność stopu (zastosowanie półniezmiennika) . . . . .                                                                                                         | 9        |
| 3.6.3    | Wniosek końcowy . . . . .                                                                                                                                       | 9        |
| <b>4</b> | <b>Implementacja wymyślonych algortymów w wybranym środowisku i języku oraz eksperymentalne potwierdzenie wydajności (złożoności obliczeniowej) algorytmów.</b> | <b>9</b> |
| 4.1      | Prosta implementacja . . . . .                                                                                                                                  | 9        |
| 4.2      | Testy „niewygodnych” zestawów danych . . . . .                                                                                                                  | 11       |
| 4.3      | Testy wydajności algorytmów - eksperymentalne sprawdzenie złożoności czasowej . . . . .                                                                         | 12       |
| 4.4      | Testy wydajności algorytmów - złożoności optymistyczne/pesymistyczne . . . . .                                                                                  | 15       |

# 1 Treść zadania

Dla zadanego ciągu liczb naturalnych znajdź najdłuższy podciąg, którego suma jest mniejsza niż zadana liczba  $k$ .

**Przykład.**

**Wejście:**  $A[] = [1, 130, 2, 5, 1, 1, 4, 4, 1, 3, 1, 1]$ ,  $k = 10$

**Wyjście:** Szukany podciąg to:  
[2, 5, 1, 1], [4, 1, 3, 1], [1, 3, 1, 1]

**Wejście:**  $A[] = [1, 130, 1, 9, 11, 6, 1, 1, 1, 3, 1, 1]$ ,  $k = 4$

**Wyjście:** Szukany podciąg to:  
[1, 1, 1]

## 2 Rozwiązanie - podejście pierwsze (brute force)

### 2.1 Analiza problemu

W pierwszym podejściu do rozwiązania zadania postąpimy metodą "siłową" (brute force), implementując rozwiązanie, które w sposób naturalny narzuca się przy analizie treści problemu. Zgodnie z definicją zadania, najdłuższy podciąg spełniający warunek sumy mniejszej od zadanej liczby  $k$  musi być podciągiem spójnym, czyli takim, który składa się z kolejnych elementów tablicy. Oznacza to że każdy podciąg można opisać za pomocą pary indeksów (początku i końca). Początek  $\leq$  Koniec.

Najprostrzym podejściem do rozwiązania problemu jest sprawdzenie wszystkich możliwych par indeksów  $(i, j)$  i obliczenie sumy elementów podciągu  $A[i..j]$ . Dla każdej takiej pary sprawdzamy, czy suma rozpatrywanego podciągu jest mniejsza od  $k$ . Jeżeli warunek ten jest spełniony, obliczamy długość podciągu jako  $j-i+1$  i porównujemy ją z aktualnie zapamiętaną długością najdłuższego poprawnego podciągu. Jeżeli nowo obliczona długość jest większa, aktualizujemy zapamiętany wynik.

Należy zwrócić uwagę na fakt, że ciąg wejściowy składa się z liczb naturalnych, a więc suma elementów podciągu jest funkcją niemalejącą wraz ze wzrostem indeksu  $j$ . Oznacza to, że w momencie, gdy suma stanie się większa lub równa  $k$ , dalsze rozszerzenie podciągu nie ma sensu. Z tego powodu w algorytmie można zastosować instrukcję przerywania pętli wewnętrznej, co pozwala ograniczyć liczbę niepotrzebnych obliczeń.

Algorytm powinien również poprawnie uwzględnić przypadki brzegowe. Jeżeli tablica wejściowa jest pusta lub zawiera tylko jeden element, algorytm nie znajdzie podciągu dłuższego niż jeden element, a zmienna najdłuższy pozostanie równa 0 lub 1, w zależności od spełnienia warunku sumy. Zwracany wynik będzie wówczas pustą listą lub jednoelementowym podciągiem, co jest zgodne z treścią zadania.

#### 1. Dane wejściowe

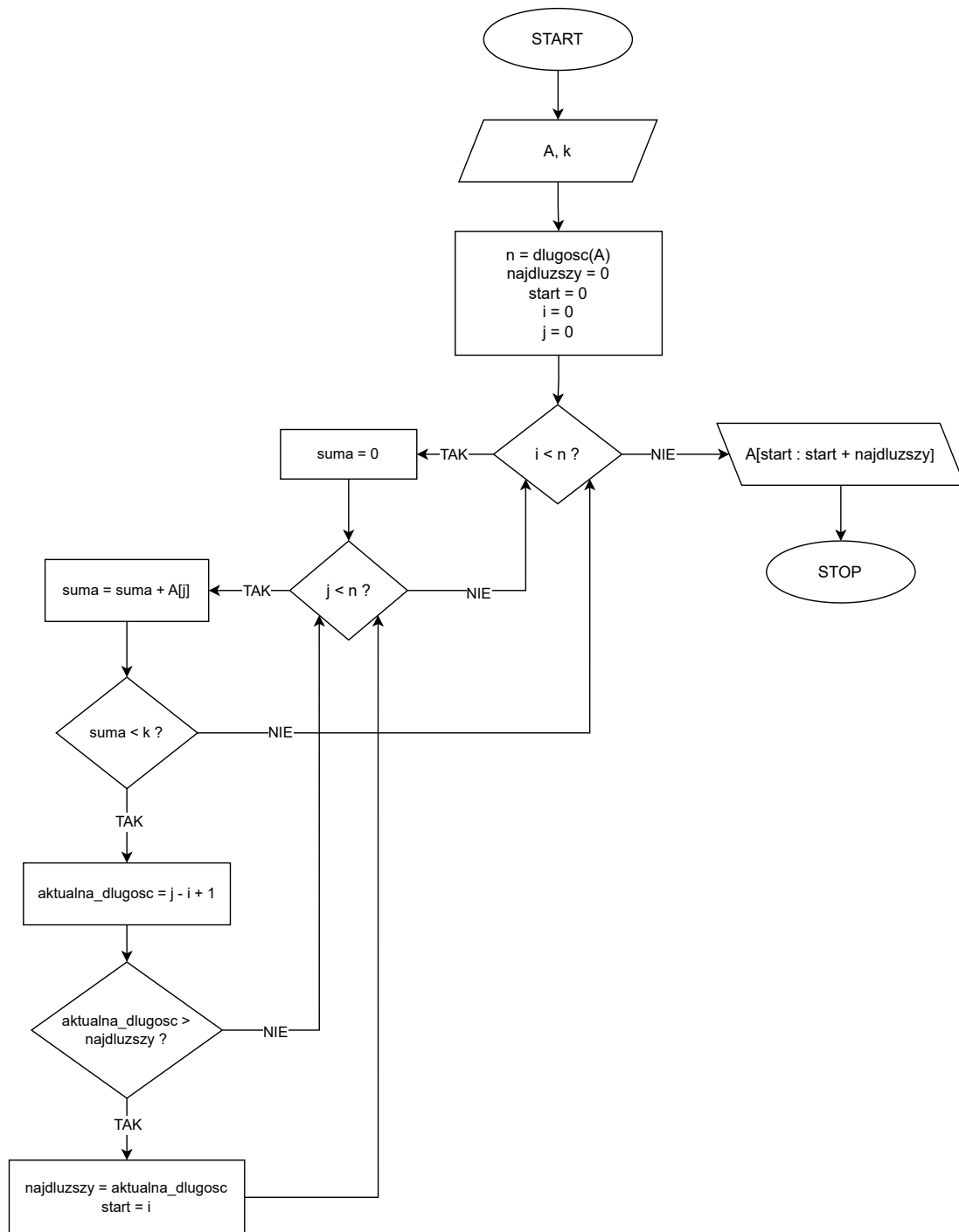
Danymi wejściowymi algorytmu są:

- tablica  $A$  zawierająca  $n$  liczb naturalnych,
- liczba naturalna  $k$ , określająca górne ograniczenie sumy podciągu.

#### 2. Dane wyjściowe

Danymi wyjściowymi algorytmu jest podciąg tablicy  $A$  o maksymalnej długości, którego suma elementów jest mniejsza niż  $k$ .

## 2.2 Schemat blokowy



Rysunek 1: Schemat blokowy - brute force

## 2.3 Pseudokod

```
1 input:  A    // tablica przechowujaca wartosci ciagu
2         k    // zadana liczba
```

```

3  output: najdluzszy podciag tablicy A o sumie < k
4
5  n := dlugosc(A)
6  najdluzszy := 0
7  start := 0
8
9  DLA i OD 0 DO n-1
10     suma := 0
11     DLA j OD i DO n-1
12         suma := suma + A[j]
13
14         JEZELI suma < k
15             aktualna_dlugosc := j - i + 1
16             JEZELI aktualna_dlugosc > najdluzszy
17                 najdluzszy := aktualna_dlugosc
18                 start := i
19         INACZEJ
20             PRZERWIJ
21
22  ZWROC A[OD start DO start + najdluzszy)

```

Listing 1: Pseudokod - brute force

## 2.4 Sprawdzenie poprawności algorytmu - „ołówkowe” rozwiązanie problemu

Analiza 'ołówkowa' algorytmu zostanie przeprowadzona na przykładzie danych wejściowych:

$A = [2, 3, 1, 15, 2, 2, 5, 12, 1, 1, 4]$ ,  $k = 8$

| lewy | prawy | A[prawy] | suma | aktualna_dlugosc | najdluzszy | start |
|------|-------|----------|------|------------------|------------|-------|
| 0    | 0     | 2        | 2    | 1                | 1          | 0     |
| 0    | 1     | 3        | 5    | 2                | 2          | 0     |
| 0    | 2     | 1        | 6    | 3                | 3          | 0     |
| 1    | 1     | 3        | 3    | 1                | 3          | 0     |
| 1    | 2     | 1        | 4    | 2                | 3          | 0     |
| 2    | 2     | 1        | 1    | 1                | 3          | 0     |
| 4    | 4     | 2        | 2    | 1                | 3          | 0     |
| 4    | 5     | 2        | 4    | 2                | 3          | 0     |
| 5    | 5     | 2        | 2    | 1                | 3          | 0     |
| 5    | 6     | 5        | 7    | 2                | 3          | 0     |
| 6    | 6     | 5        | 5    | 1                | 3          | 0     |
| 8    | 8     | 1        | 1    | 1                | 3          | 0     |
| 8    | 9     | 1        | 2    | 2                | 3          | 0     |
| 8    | 10    | 4        | 6    | 3                | 3          | 0     |
| 9    | 9     | 1        | 1    | 1                | 3          | 0     |
| 9    | 10    | 4        | 5    | 2                | 3          | 0     |
| 10   | 10    | 4        | 4    | 1                | 3          | 0     |

Tabela 1: Ołówkowe śledzenie algorytmu brute-force

## 2.5 Teoretyczne oszacowanie złożoności obliczeniowej

Podstawową operacją jest dodanie elementu do sumy i sprawdzenie warunku  $\text{suma} < k$ . W najgorszym przypadku (jeśli suma rzadko przekracza  $k$ ) wewnętrzna pętla przebiega do końca dla każdego  $i$ .

- dla  $i=0$ :  $n$  operacji
- dla  $i=1$ :  $n-1$  operacji
- dla  $i=2$ :  $n-2$  operacji
- ...

- dla  $i=n-1$ : 1 operacja

$$\sum_{k=1}^n k = \frac{n(n+1)}{2} \quad (1)$$

Zatem złożoność czasowa w najgorszym przypadku wynosi:

$$O(n^2) \quad (2)$$

## 2.6 Wykazanie poprawności algorytmu

### 2.6.1 Poprawność częściowa (zastosowanie niezmiennika)

Rozważanym problemem jest znalezienie najdłuższego podciągu tablicy  $A$  o sumie elementów mniejszej niż  $k$ .

Jako niezmiennik pętli zewnętrznej (po indeksie  $i$ ) przyjmujemy stwierdzenie:

Po zakończeniu każdej iteracji zmienne **najdluzszy** i **start** przechowują długość oraz początek najdłuższego podciągu spośród wszystkich podciągów zaczynających się w indeksach  $0..i$  o sumie mniejszej niż  $k$ .

- Przed pierwszą iteracją: **najdluzszy** = 0, **start** = 0 – niezmiennik prawdziwy.
- Podczas iteracji: wewnętrzna pętla po  $j$  sumuje elementy od  $i$  do  $j$  i aktualizuje zmienne tylko jeśli suma  $< k$  oraz długość podciągu jest większa od **najdluzszy**.
- Po zakończeniu pętli zewnętrznej: wszystkie możliwe podciągi zostały rozpatrzone, zatem niezmiennik wskazuje warunek końcowy algorytmu.

### 2.6.2 Własność stopu (zastosowanie półniezmiennika)

Algorytm zawiera dwie pętle: zewnętrzną po  $i$  i wewnętrzną po  $j$ .

#### Pętla wewnętrzna

$$V_1(j) = n - j$$

- $V_1(j) \in \mathbb{N}$
- W każdej iteracji  $j$  rośnie o 1, więc  $V_1(j)$  maleje
- Dolna granica:  $V_1(j) \geq 0$ ; dla  $j = n$  pętla kończy działanie

#### Pętla zewnętrzna

$$V_2(i) = n - i$$

- $V_2(i) \in \mathbb{N}$
- W każdej iteracji  $i$  rośnie o 1, więc  $V_2(i)$  maleje
- Dolna granica:  $V_2(i) \geq 0$ ; dla  $i = n$  pętla kończy działanie

### 2.6.3 Wniosek końcowy

Skoro algorytm spełnia poprawność częściową (niezmiennik) oraz posiada własność stopu (półniezmienniki dla obu pętli), algorytm jest poprawny całkowicie.

## 3 Rozwiązanie - próba druga (sliding window)

### 3.1 Analiza problemu

Po zapoznaniu się z rozwiązaniem brute force można zauważyć, że jego główną wadą jest konieczność wielokrotnego obliczania sum tych samych fragmentów tablicy. Dla każdego nowego początku podciągu suma elementów jest liczona od zera, mimo że wiele z tych operacji się powtarza. Prowadzi to do złożoności kwadratowej, która dla długich ciągów staje się nieefektywna. W celu usprawnienia algorytmu należy zatem poszukać metody, która pozwoli wykorzystywać wcześniej wykonane obliczenia.

Kluczową obserwacją jest fakt, że rozpatrywany ciąg składa się z liczb naturalnych. Oznacza to, że jeżeli do aktualnego podciągu dołożymy kolejny element z prawej strony, to jego suma nie zmniejszy się, lecz co najwyżej wzrośnie. Własność ta sugeruje możliwość utrzymywania jednego „ruchomego” przedziału elementów tablicy, którego granice będą zmieniane w trakcie działania algorytmu, zamiast każdorazowego rozpoczynania obliczeń od nowa.

Możemy stworzyć tzw. okno z dwoma indeksami (lewy oraz prawy). Indeks lewy wskazuje początek aktualnie analizowanego podciągu, natomiast prawy – jego koniec. Na początku algorytmu oba indeksy ustawiamy na początek tablicy. Dodatkowo używamy zmiennej suma, która będzie oznaczać sumę aktualnie analizowanego podciągu.

Następnie można przesuwając indeks prawy w prawo, dołączając kolejne elementy tablicy do aktualnego podciągu i zwiększając sumę o wartość nowo dodanego elementu. Po każdym rozszerzeniu okna sprawdzamy, czy suma elementów podciągu nadal spełnia warunek  $< k$ . Gdy suma jest większa bądź równa  $k$ , należy skrócić podciąg.

Skracanie podciągu realizujemy poprzez zwiększenie indeksu lewy i odjęcie od sumy elementu, który usunęliśmy. Operację powtarzamy tak długo aż suma ponownie będzie mniejsza od  $k$ .

#### 1. Dane wejściowe

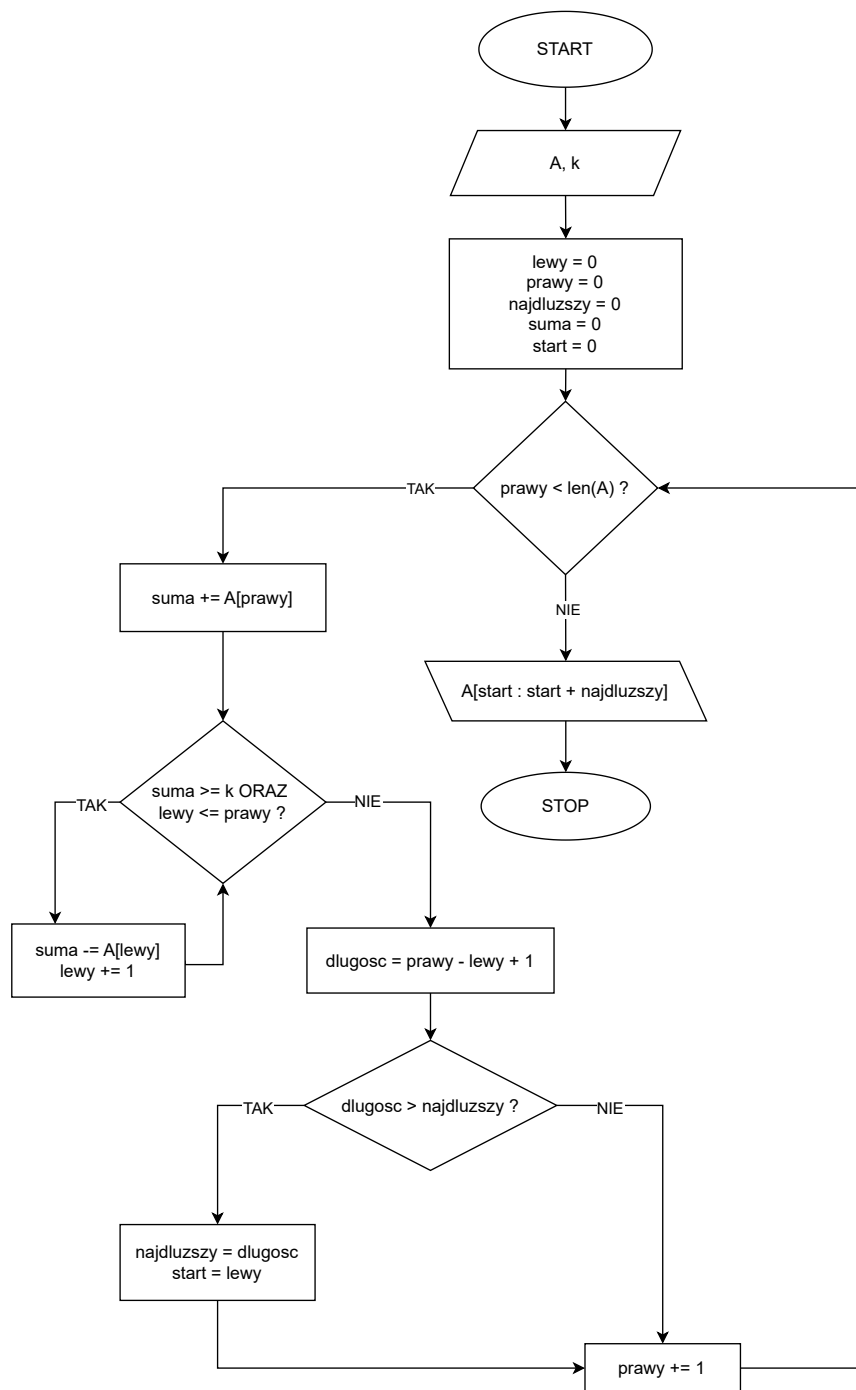
Danymi wejściowymi algorytmu są:

- tablica  $A$  zawierająca  $n$  liczb naturalnych,
- liczba naturalna  $k$ , określająca górne ograniczenie sumy podciągu.

#### 2. Dane wyjściowe

Danymi wyjściowymi algorytmu jest podciąg tablicy  $A$  o maksymalnej długości, którego suma elementów jest mniejsza niż  $k$ .

### 3.2 Schemat blokowy



Rysunek 2: Schemat blokowy - sliding window

### 3.3 Pseudokod

```
1 input:  A    // tablica przechowująca wartości ciągu
2         k    // zadana liczba
```



```

3  output: najdluzszy podciag tablicy A o sumie < k
4
5  lewy := 0
6  najdluzszy := 0
7  start := 0
8  suma := 0
9
10 DLA prawy OD 0 DO dlugosc(A) - 1
11     suma := suma + A[prawy]
12
13     DOPOKI suma >= k ORAZ lewy <= prawy
14         suma := suma - A[lewy]
15         lewy := lewy + 1
16
17     aktualna_dlugosc := prawy - lewy + 1
18
19     JEZELI aktualna_dlugosc > najdluzszy
20         najdluzszy := aktualna_dlugosc
21         start := lewy
22
23 ZWROC A[OD start DO start + najdluzszy)

```

Listing 2: Pseudokod - sliding window

### 3.4 Sprawdzenie poprawności algorytmu - „ołówkowe” rozwiązanie problemu

Analiza 'ołówkowa' algorytmu zostanie przeprowadzona na przykładzie danych wejściowych:

$A = [2, 3, 1, 15, 2, 2, 5, 12, 1, 1, 4]$ ,  $k = 8$

| prawy | A[prawy] | suma | lewy | aktualna_dlugosc | najdluzszy | start |
|-------|----------|------|------|------------------|------------|-------|
| 0     | 2        | 2    | 0    | 1                | 1          | 0     |
| 1     | 3        | 5    | 0    | 2                | 2          | 0     |
| 2     | 1        | 6    | 0    | 3                | 3          | 0     |
| 3     | 15       | 0    | 4    | 0                | 3          | 0     |
| 4     | 2        | 2    | 4    | 1                | 3          | 0     |
| 5     | 2        | 4    | 4    | 2                | 3          | 0     |
| 6     | 5        | 7    | 5    | 2                | 3          | 0     |
| 7     | 12       | 0    | 8    | 0                | 3          | 0     |
| 8     | 1        | 1    | 8    | 1                | 3          | 0     |
| 9     | 1        | 2    | 8    | 2                | 3          | 0     |
| 10    | 4        | 6    | 8    | 3                | 3          | 0     |

Tabela 2: Ołówkowe śledzenie algorytmu sliding window

### 3.5 Teoretyczne oszacowanie złożoności obliczeniowej

Każdy element jest dodawany do sumy dokładnie raz. Każdy element jest odejmowany od sumy dokładnie raz w momencie przesunięcia lewego końca okna. Zatem łączna liczba operacji dodawania i odejmowania jest mniejsze bądź równe  $2n$ . Nie ma zagnieżdżonych pętli, które w pełni przeglądają tablicę.

$$n + n = 2n \quad (3)$$

Zatem złożoność czasowa wynosi:

$$O(n) \quad (4)$$

### 3.6 Wykazanie poprawności algorytmu

#### 3.6.1 Poprawność częściowa (zastosowanie niezmiennika)

Rozważanym problemem jest znalezienie najdłuższego podciągu tablicy  $A$  o sumie elementów mniejszej niż  $k$ .

Jako niezmiennik pętli po indeksie **prawy** przyjmujemy stwierdzenie:

Po zakończeniu każdej iteracji zmienne **najdluzszy**, **start** oraz **lewy** spełniają warunki: zmienna **najdluzszy** przechowuje długość najdłuższego podciągu spośród wszystkich podciągów kończących się na indeksach  $0..prawy$  o sumie elementów  $< k$ , a **start** przechowuje początek tego podciągu. Zmienna **lewy** wskazuje aktualny początek rozważanego podciągu z sumą  $< k$ .

- Przed pierwszą iteracją: **najdluzszy** = 0, **start** = 0, **lewy** = 0 – niezmiennik prawdziwy.
- Podczas iteracji: dodanie elementu  $A[prawy]$  do zmiennej **suma** oraz przesuwanie wskaźnika **lewy** w pętli **while** zapewnia, że suma bieżącego podciągu jest  $< k$ . Następnie aktualizowane są zmienne **najdluzszy** i **start**, jeśli aktualna długość jest większa.
- Po zakończeniu pętli: wszystkie podciągi zostały rozpatrzone zgodnie z warunkiem, zatem niezmiennik implikuje poprawny wynik algorytmu.

#### 3.6.2 Własność stopu (zastosowanie półniezmiennika)

Algorytm zawiera dwie pętle: główną po **prawy** i wewnętrzną po **lewy**.

**Pętla wewnętrzna**

$$V_1(lewy) = prawy - lewy + 1$$

- $V_1(lewy) \in \mathbb{N}$
- W każdej iteracji **lewy** zwiększa się o 1, więc  $V_1(lewy)$  maleje
- Dolna granica:  $V_1(lewy) \geq 0$ ; gdy warunek **suma**  $\geq k$  nie jest spełniony lub **lewy**  $>$  **prawy**, pętla przestaje działać

**Pętla zewnętrzna**

$$V_2(prawy) = n - prawy, \quad n = |A|$$

- $V_2(prawy) \in \mathbb{N}$
- W każdej iteracji **prawy** rośnie o 1, więc  $V_2(prawy)$  maleje
- Dolna granica:  $V_2(prawy) \geq 0$ ; dla **prawy** =  $n$  pętla kończy działanie

#### 3.6.3 Wniosek końcowy

Skoro algorytm spełnia poprawność częściową (niezmiennik pętli) oraz posiada własność stopu (półniezmienniki dla obu pętli), algorytm jest poprawny całkowicie.

## 4 Implementacja wymyślonych algorytmów w wybranym środowisku i języku oraz eksperymentalne potwierdzenie wydajności (złożoności obliczeniowej) algorytmów.

### 4.1 Prosta implementacja

```

1  #include <iostream>
2  #include <vector>
3
4  using namespace std;
5
6  vector<vector<int>> brute_force(int A[], int k, int n) {
7      int najdluzszy = 0;
8      vector<int> starts;
9
10     for(int i = 0; i < n; i++){
11         int suma = 0;
12         for(int j = i; j < n; j++){
13             suma += A[j];
14
15             if(suma < k){
16                 int aktualna_dlugosc = j - i + 1;
17
18                 if(aktualna_dlugosc > najdluzszy){
19                     najdluzszy = aktualna_dlugosc;
20                     starts.clear();
21                     starts.push_back(i);
22                 }
23                 else if(aktualna_dlugosc == najdluzszy){
24                     starts.push_back(i);
25                 }
26             }
27             else{
28                 break;
29             }
30         }
31     }
32
33     vector<vector<int>> result;
34     for(int i = 0; i < starts.size(); i++){
35         result.push_back(vector<int>(A + starts[i], A + starts[i] + najdluzszy));
36     }
37
38     return result;
39 }
40
41 vector<vector<int>> sliding_window(int A[], int k, int n) {
42     int lewy = 0;
43     int najdluzszy = 0;
44     vector<int> starts;
45     int suma = 0;
46
47     for(int prawy = 0; prawy < n; prawy++){
48         suma += A[prawy];
49
50         while(suma >= k && lewy <= prawy){
51             suma -= A[lewy];
52             lewy++;
53         }
54
55         int aktualna_dlugosc = prawy - lewy + 1;
56         if(aktualna_dlugosc > najdluzszy){
57             najdluzszy = aktualna_dlugosc;
58             starts.clear();
59             starts.push_back(lewy);
60         }
61         else if(aktualna_dlugosc == najdluzszy){
62             starts.push_back(lewy);
63         }

```

```

64     }
65
66     vector<vector<int>> result;
67     for(int i = 0; i < starts.size(); i++){
68         result.push_back(vector<int>(A + starts[i], A + starts[i] + najdluzszy));
69     }
70
71     return result;
72 }
73
74 void wypisz_resultat(vector<vector<int>> result) {
75     for (int i = 0; i < result.size(); i++){
76         cout << "[";
77         for(int j = 0; j < result[i].size(); j++){
78             cout << result[i][j] << " ";
79         }
80         cout << "]" ";
81     }
82     cout << endl;
83 }
84
85 int main() {
86     int A[] = {1, 130, 2, 5, 1, 1, 4, 4, 1, 3, 1, 1};
87     int n = sizeof(A) / sizeof(A[0]);
88     int k = 10;
89
90     cout << "Brute force: ";
91     wypisz_resultat(brute_force(A, k, n));
92
93     cout << "Sliding window: ";
94     wypisz_resultat(sliding_window(A, k, n));
95
96     return 0;
97 }

```

Brute force: [2 5 1 1] [4 1 3 1] [1 3 1 1]  
 Sliding window: [2 5 1 1] [4 1 3 1] [1 3 1 1]

## 4.2 Testy „niewygodnych” zestawów danych

```

1  int main() {
2      int A1[] = {1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1};
3      int A2[] = {10, 20, 30, 40, 50, 60};
4      int A3[] = {7, -3, 6, -4, 9};
5      int A4[] = {1, 100, 2, 90, 3, 80, 4, 70};
6
7      int k1 = 4;
8      int k2 = 5;
9      int k3 = 10;
10     int k4 = 50;
11
12     int n1 = sizeof(A1) / sizeof(A1[0]);
13     int n2 = sizeof(A2) / sizeof(A2[0]);
14     int n3 = sizeof(A3) / sizeof(A3[0]);
15     int n4 = sizeof(A4) / sizeof(A4[0]);
16
17     cout << "Brute force: " << endl;
18     cout << "Test 1: ";
19     wypisz_resultat(brute_force(A1, k1, n1));
20     cout << "Test 2: ";
21     wypisz_resultat(brute_force(A2, k2, n2));
22     cout << "Test 3: ";
23     wypisz_resultat(brute_force(A3, k3, n3));

```

```

24     cout << "Test 4: ";
25     wypisz_resultat(brute_force(A4, k4, n4));
26
27     cout << endl;
28
29     cout << "Sliding window: " << endl;
30     cout << "Test 1: ";
31     wypisz_resultat(sliding_window(A1, k1, n1));
32     cout << "Test 2: ";
33     wypisz_resultat(sliding_window(A2, k2, n2));
34     cout << "Test 3: ";
35     wypisz_resultat(sliding_window(A3, k3, n3));
36     cout << "Test 4: ";
37     wypisz_resultat(sliding_window(A4, k4, n4));
38
39     return 0;
40 }

```

Brute force:

Test 1: [1 1 1] [1 1 1] [1 1 1] [1 1 1] [1 1 1] [1 1 1] [1 1 1] [1 1 1] [1 1 1] [1 1 1] [1 1 1] [1 1 1] [1 1 1] [1 1 1] [1 1 1] [1 1 1] [1 1 1] [1 1 1] [1 1 1] [1 1 1]

Test 2:

Test 3: [-3 6 -4 9]

Test 4: [1] [2] [3] [4]

Sliding window:

Test 1: [1 1 1] [1 1 1] [1 1 1] [1 1 1] [1 1 1] [1 1 1] [1 1 1] [1 1 1] [1 1 1] [1 1 1] [1 1 1] [1 1 1] [1 1 1] [1 1 1] [1 1 1] [1 1 1] [1 1 1] [1 1 1] [1 1 1] [1 1 1]

Test 2: [] [] [] [] [] []

Test 3: [-3 6 -4 9]

Test 4: [1] [2] [3] [4]

Gdy mamy dużą tablicę z małymi wartościami w porównaniu do k. Otrzymamy bardzo dużo podobnych tablic.

### 4.3 Testy wydajności algorytmów - eksperymentalne sprawdzenie złożoności czasowej

```

1  #include <iostream>
2  #include <vector>
3
4  using namespace std;
5
6  vector<vector<int>> brute_force(int A[], int k, int n) {
7      int najdluzszy = 0;
8      vector<int> starts;
9
10     for(int i = 0; i < n; i++){
11         int suma = 0;
12         for(int j = i; j < n; j++){
13             suma += A[j];
14
15             if(suma < k){
16                 int aktualna_dlugosc = j - i + 1;
17
18                 if(aktualna_dlugosc > najdluzszy){
19                     najdluzszy = aktualna_dlugosc;
20                     starts.clear();
21                     starts.push_back(i);
22                 }
23                 else if(aktualna_dlugosc == najdluzszy){
24                     starts.push_back(i);

```

```

25         }
26     }
27     else{
28         break;
29     }
30 }
31 }
32
33 vector<vector<int>> result;
34 for(int i = 0; i < starts.size(); i++){
35     result.push_back(vector<int>(A + starts[i], A + starts[i] + najdluzszy));
36 }
37
38 return result;
39 }
40
41 vector<vector<int>> sliding_window(int A[], int k, int n) {
42     int lewy = 0;
43     int najdluzszy = 0;
44     vector<int> starts;
45     int suma = 0;
46
47     for(int prawy = 0; prawy < n; prawy++){
48         suma += A[prawy];
49
50         while(suma >= k && lewy <= prawy){
51             suma -= A[lewy];
52             lewy++;
53         }
54
55         int aktualna_dlugosc = prawy - lewy + 1;
56         if(aktualna_dlugosc > najdluzszy){
57             najdluzszy = aktualna_dlugosc;
58             starts.clear();
59             starts.push_back(lewy);
60         }
61         else if(aktualna_dlugosc == najdluzszy){
62             starts.push_back(lewy);
63         }
64     }
65
66     vector<vector<int>> result;
67     for(int i = 0; i < starts.size(); i++){
68         result.push_back(vector<int>(A + starts[i], A + starts[i] + najdluzszy));
69     }
70
71     return result;
72 }
73
74 void generuj(int *tab, int n, int nmax) {
75     for(int i = 0; i < n; i++){
76         tab[i] = rand() % nmax;
77     }
78 }
79
80 void wypisz_resultat(vector<vector<int>> result) {
81     for (int i = 0; i < result.size(); i++){
82         cout << "[";
83         for(int j = 0; j < result[i].size(); j++){
84             cout << result[i][j] << " ";
85         }
86         cout << "]" ";
87     }

```

```

88     cout << endl;
89 }
90
91 int *tab;
92 float *czas1;
93 float *czas2;
94
95 clock_t start, finish;
96
97 int main() {
98     int N[] = {250000, 500000, 1000000, 2000000, 3000000, 4000000, 5000000, 6000000,
99               7000000, 8000000};
100    int liczba_testow = sizeof(N) / sizeof(N[0]);
101    czas1 = (float *) malloc(liczba_testow * sizeof (float) );
102    czas2 = (float *) malloc(liczba_testow * sizeof (float) );
103
104    for(int i = 0; i < liczba_testow; i++){
105        tab = (int *) malloc(N[i] * sizeof (int) );
106        generuj(tab, N[i], 10000);
107
108        start = clock();
109        brute_force(tab, N[i]/2, N[i]);
110        finish = clock();
111        czas1[i] = ((float)(finish - start)) / CLOCKS_PER_SEC;
112
113        start = clock();
114        sliding_window(tab, N[i]/2, N[i]);
115        finish = clock();
116        czas2[i] = ((float)(finish - start)) / CLOCKS_PER_SEC;
117        free(tab);
118    }
119    cout << endl << endl;
120    printf(" L.p.      n      t1 [s]      t2 [s] \n ");
121    for(int i = 0; i < liczba_testow; i++){
122        printf("%2d %10d %6.6f %6.6f\n ", i+1, N[i], czas1[i], czas2[i]) ;
123    }
124
125    return 0;
126 }

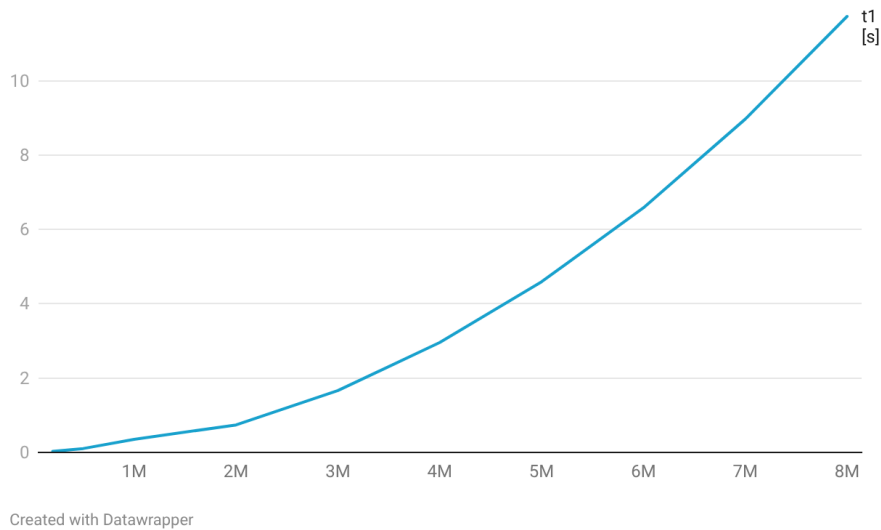
```

| L.p. | n       | $t_1$ [s] | $t_2$ [s] |
|------|---------|-----------|-----------|
| 1    | 200000  | 0.031003  | 0.003935  |
| 2    | 500000  | 0.104805  | 0.007289  |
| 3    | 1000000 | 0.353430  | 0.007711  |
| 4    | 2000000 | 0.739041  | 0.014497  |
| 5    | 3000000 | 1.663470  | 0.021500  |
| 6    | 4000000 | 2.957843  | 0.028472  |
| 7    | 5000000 | 4.590719  | 0.035083  |
| 8    | 6000000 | 6.581690  | 0.042666  |
| 9    | 7000000 | 8.967931  | 0.049816  |
| 10   | 8000000 | 11.732619 | 0.057095  |

Tabela 3: Czasy wykonania algorytmów

### Czas obliczeń dla brute force

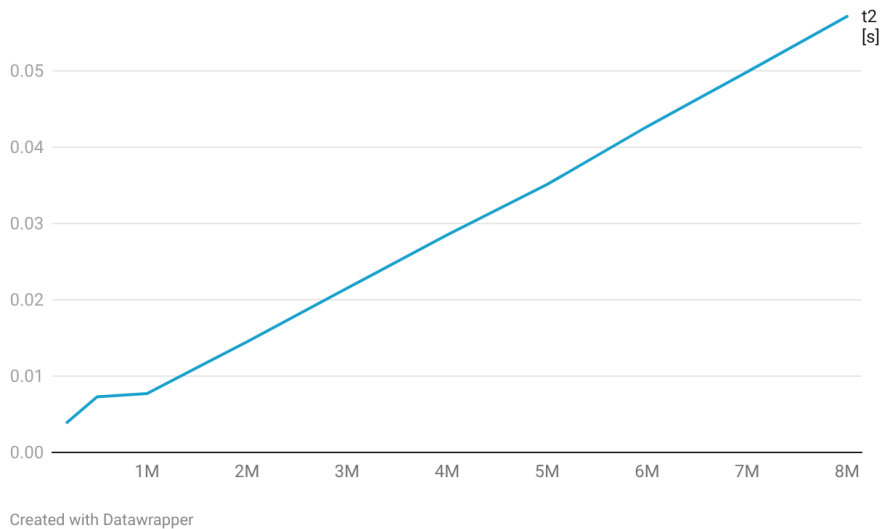
$$t(n) = 1.4274e-06 \cdot n - 1.4664$$



Rysunek 3: Wykres złożoności brute force

### Czas obliczeń dla sliding window

$$t(n) = 6.7903e-09 \cdot n + 1.8859e-03$$



Rysunek 4: Wykres złożoności sliding window

## 4.4 Testy wydajności algorytmów - złożoności optymistyczne/pesymistyczne

W przypadku tego problemu im mniejsze k tym szybciej algorytm będzie się wykonywał i tym mniejsza różnica w wydajności będzie między tymi dwoma algorytmami.

```
1 int main() {
2     // Testowanie złożoności
3     int N[] = {250000, 500000, 1000000, 2000000, 3000000, 4000000, 5000000, 6000000,
4               7000000, 8000000};
5     int liczba_testow = sizeof(N) / sizeof(N[0]);
6     czas1 = (float *) malloc(liczba_testow * sizeof(float));
7     czas2 = (float *) malloc(liczba_testow * sizeof(float));
```



```

7
8   for(int i = 0; i < liczba_testow; i++){
9       tab = (int *) malloc(N[i] * sizeof (int) );
10      generuj(tab, N[i], 10000);
11
12      start = clock();
13      brute_force(tab, N[i]/20, N[i]);
14      finish = clock();
15      czas1[i] = ((float)(finish - start)) / CLOCKS_PER_SEC;
16
17      start = clock();
18      sliding_window(tab, N[i]/20, N[i]);
19      finish = clock();
20      czas2[i] = ((float)(finish - start)) / CLOCKS_PER_SEC;
21      free(tab);
22  }
23  cout << endl << endl;
24  printf(" L.p.      n      t1 [s]      t2 [s] \n ");
25  for(int i = 0; i < liczba_testow; i++){
26      printf("%2d %10d %6.6f %6.6f\n", i+1, N[i], czas1[i], czas2[i]) ;
27  }
28
29  return 0;
30 }

```

| Lp. | $n$    | $t_1$ [s] | $t_2$ [s] |
|-----|--------|-----------|-----------|
| 1   | 250000 | 0.004795  | 0.004189  |
| 2   | 50000  | 0.017081  | 0.007819  |
| 3   | 100000 | 0.054856  | 0.015911  |
| 4   | 200000 | 0.182500  | 0.028738  |
| 5   | 300000 | 0.226510  | 0.020939  |
| 6   | 400000 | 0.314604  | 0.028059  |
| 7   | 500000 | 0.485833  | 0.035116  |
| 8   | 600000 | 0.690309  | 0.042426  |
| 9   | 700000 | 0.947294  | 0.049680  |
| 10  | 800000 | 1.217408  | 0.057858  |

Tabela 4: Czasy wykonania algorytmów

## Spis rysunków

|   |                                  |    |
|---|----------------------------------|----|
| 1 | Schemat blokowy - brute force    | 3  |
| 2 | Schemat blokowy - sliding window | 7  |
| 3 | Wykres złożoności brute force    | 15 |
| 4 | Wykres złożoności sliding window | 15 |

## Spis tabel

|   |                                             |    |
|---|---------------------------------------------|----|
| 1 | Ołówkowe śledzenie algorytmu brute-force    | 4  |
| 2 | Ołówkowe śledzenie algorytmu sliding window | 8  |
| 3 | Czasy wykonania algorytmów                  | 14 |
| 4 | Czasy wykonania algorytmów                  | 16 |